

# MAP4002-Otim Linear\_L1\_v1

September 6, 2025

## 1 Questão 4 — Produção de vigas (PL com Gurobi)

### 1.1 Luis Alvaro Correia (nUSP 745724)

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import gurobipy as gp
from gurobipy import Model, GRB, quicksum
```

```
[2]: # -----
# Dados do problema
# -----
T = ["p", "m", "g", "e"]          # tipos: pequena, mediana, grande, extra
M = ["A", "B", "C"]              # máquinas

# taxas  $r_{\{t,m\}}$  (metros por hora) conforme a tabela
r = {
    "p": {"A": 350, "B": 650, "C": 850},
    "m": {"A": 250, "B": 400, "C": 700},
    "g": {"A": 200, "B": 350, "C": 600},
    "e": {"A": 125, "B": 200, "C": 325},
}

# demandas (metros)
d = {"p": 12000, "m": 6000, "g": 5000, "e": 7000}

# custo por hora por máquina
c = {"A": 30, "B": 50, "C": 80}

# horas disponíveis por máquina
H = {"A": 50, "B": 50, "C": 50}

# -----
# Modelo
# -----
```

```

m = Model("steel_beams")
m.Params.OutputFlag = 1 # 1 para log, 0 para silêncio

# variáveis: horas dedicadas a cada (tipo, máquina)
h = m.addVars(T, M, name="h", lb=0.0, vtype=GRB.CONTINUOUS)

# objetivo: minimizar custo total por hora
m.setObjective(quicksum(c[j] * quicksum(h[t, j] for t in T) for j in M), GRB.
↳MINIMIZE)

# demanda por tipo (produção >= demanda)
dem_ctr = {}
for t in T:
    dem_ctr[t] = m.addConstr(quicksum(r[t][j] * h[t, j] for j in M) >= d[t],
↳name=f"dem_{t}")

# limite de horas por máquina
cap_ctr = {}
for j in M:
    cap_ctr[j] = m.addConstr(quicksum(h[t, j] for t in T) <= H[j],
↳name=f"cap_{j}")

# opcional: gravar o LP
#m.write("steel_beams.lp")

# otimiza
m.optimize()

# -----
# Relatórios / Saída formatada
# -----
if m.SolCount > 0:
    # Tabela de horas por (tipo, máquina)
    hours_data = {(t,j): h[t,j].X for t in T for j in M}
    hours_df = pd.Series(hours_data).unstack().reindex(index=T, columns=M)
    hours_df.index = ["pequena (p)", "mediana (m)", "grande (g)", "extra (e)"]
    hours_df.columns = [f"Maq {j}" for j in M]

    # Produção por tipo (soma nas máquinas) e por (tipo, máquina)
    prod_data = {(t,j): r[t][j] * h[t,j].X for t in T for j in M}
    prod_df = pd.Series(prod_data).unstack().reindex(index=T, columns=M)
    prod_df.index = hours_df.index
    prod_df.columns = [f"Maq {j}" for j in M]
    prod_df["Total (m)"] = prod_df.sum(axis=1)
    prod_req = pd.Series({t: d[t] for t in T}, index=T)
    prod_df["Demanda (m)"] = prod_req.values
    prod_df["Excesso (m)"] = prod_df["Total (m)"] - prod_df["Demanda (m)"]

```

```

# Uso total de horas por máquina
use_hours = pd.Series({j: sum(h[t,j].X for t in T) for j in M})
use_hours.index = [f"Maq {j}" for j in M]
cap_hours = pd.Series({f"Maq {j}": H[j] for j in M})
slack_hours = cap_hours - use_hours

# Objetivo
total_cost = m.ObjVal

print("\n===== OBJETIVO =====")
print(f"Custo mínimo (US$): {total_cost:,.2f}")

print("\n===== HORAS POR (TIPO, MÁQUINA) =====")
#display(hours_df.style.format("{:.4f}"))
print(hours_df.to_string())

print("\n===== PRODUÇÃO (m) POR (TIPO, MÁQUINA) =====")
# display(prod_df.style.format("{:,.2f}"))
print(prod_df.to_string())

print("\n===== USO DE HORAS POR MÁQUINA =====")
use_tbl = pd.DataFrame({
    "Uso (h)": use_hours,
    "Capacidade (h)": cap_hours,
    "Folga (h)": slack_hours
})
# display(use_tbl.style.format("{:.4f}"))
print(use_tbl.to_string())

else:
    print("Modelo inviável ou sem solução encontrada.")

```

Set parameter Username

Set parameter LicenseID to value 2673838

Academic license - for non-commercial use only - expires 2026-05-31

Set parameter OutputFlag to value 1

Gurobi Optimizer version 12.0.2 build v12.0.2rc0 (win64 - Windows 11.0 (26100.2))

CPU model: Intel(R) Core(TM) i9-14900HX, instruction set [SSE2|AVX|AVX2]

Thread count: 24 physical cores, 32 logical processors, using up to 32 threads

Optimize a model with 7 rows, 12 columns and 24 nonzeros

Model fingerprint: 0x6d5d8b0b

Coefficient statistics:

Matrix range [1e+00, 9e+02]

Objective range [3e+01, 8e+01]

```

Bounds range      [0e+00, 0e+00]
RHS range         [5e+01, 1e+04]
Presolve time: 0.00s
Presolved: 7 rows, 12 columns, 24 nonzeros

```

| Iteration | Objective     | Primal Inf.  | Dual Inf.    | Time |
|-----------|---------------|--------------|--------------|------|
| 0         | 0.0000000e+00 | 1.156250e+03 | 0.000000e+00 | 0s   |
| 5         | 3.9600733e+03 | 0.000000e+00 | 0.000000e+00 | 0s   |

```

Solved in 5 iterations and 0.00 seconds (0.00 work units)
Optimal objective 3.960073260e+03

```

```

===== OBJETIVO =====

```

```

Custo mínimo (US$): 3,960.07

```

```

===== HORAS POR (TIPO, MÁQUINA) =====

```

|             | Maq A | Maq B     | Maq C    |
|-------------|-------|-----------|----------|
| pequena (p) | 0.0   | 18.461538 | 0.000000 |
| mediana (m) | 0.0   | 0.000000  | 8.571429 |
| grande (g)  | 0.0   | 0.000000  | 8.333333 |
| extra (e)   | 50.0  | 0.000000  | 2.307692 |

```

===== PRODUÇÃO (m) POR (TIPO, MÁQUINA) =====

```

|             | Maq A  | Maq B   | Maq C  | Total (m) | Demanda (m) | Excesso (m)   |
|-------------|--------|---------|--------|-----------|-------------|---------------|
| pequena (p) | 0.0    | 12000.0 | 0.0    | 12000.0   | 12000       | -1.818989e-12 |
| mediana (m) | 0.0    | 0.0     | 6000.0 | 6000.0    | 6000        | 0.000000e+00  |
| grande (g)  | 0.0    | 0.0     | 5000.0 | 5000.0    | 5000        | 0.000000e+00  |
| extra (e)   | 6250.0 | 0.0     | 750.0  | 7000.0    | 7000        | 0.000000e+00  |

```

===== USO DE HORAS POR MÁQUINA =====

```

|       | Uso (h)   | Capacidade (h) | Folga (h) |
|-------|-----------|----------------|-----------|
| Maq A | 50.000000 | 50             | 0.000000  |
| Maq B | 18.461538 | 50             | 31.538462 |
| Maq C | 19.212454 | 50             | 30.787546 |

## 2 Questão 5

```

[3]: # -----
# Parâmetros
# -----
products = ["p1", "p2"]

# lucro unitário (preço - matéria-prima)
profit = {"p1": 7.8, "p2": 7.1}
#profit = {"p1": 7.8, "p2": 10.4}

# requisitos de tempo (h/unidade)

```

```

assembly = {"p1": 1/4, "p2": 1/3} # montagem
test      = {"p1": 1/8, "p2": 1/3} # teste

# capacidades diárias (h)
cap = {"assembly": 90.0, "test": 80.0}

# -----
# Modelo
# -----
m5 = Model("q5_production")
m5.Params.OutputFlag = 1 # exibir log

# Variáveis: produção diária
x = m5.addVars(products, name="x", lb=0.0, vtype=GRB.CONTINUOUS)

# Objetivo: maximizar lucro total
m5.setObjective(quicksum(profit[p] * x[p] for p in products), GRB.MAXIMIZE)

# Restrições de capacidade
constr_assembly = m5.addConstr(quicksum(assembly[p] * x[p] for p in products)
    <= cap["assembly"],
                                name="cap_assembly")
constr_test      = m5.addConstr(quicksum(test[p] * x[p] for p in products)
    <= cap["test"],
                                name="cap_test")

# Opcional: escrever arquivo LP
m5.write("q5_production.lp")

# Otimiza
m5.optimize()

# -----
# Relatórios / Saída formatada
# -----
if m5.SolCount > 0:
    # Plano de produção
    plan = pd.Series({p: x[p].X for p in products}, name="Unidades")
    plan.index = ["Produto 1 (x1)", "Produto 2 (x2)"]

    # Uso de recursos
    use = {
        "Montagem (h)": sum(assembly[p] * x[p].X for p in products),
        "Teste (h)":    sum(test[p] * x[p].X for p in products),
    }
    use_df = pd.DataFrame({
        "Uso (h)": pd.Series(use),

```

```

        "Capacidade (h)": pd.Series({"Montagem (h)": cap["assembly"], "Teste_
↵(h)": cap["test"]})
    })
    use_df["Folga (h)"] = use_df["Capacidade (h)"] - use_df["Uso (h)"]

    # Lucro ótimo
    z_opt = m5.ObjVal

    # Impressão
    print("\n===== LUCRO ÓTIMO =====")
    print(f"z* = {z_opt:,.2f} (unidades monetárias)")

    print("\n===== PLANO DE PRODUÇÃO (x1, x2) =====")
    # display(plan.to_frame())
    print(plan.to_string())

    print("\n===== USO DE RECURSOS =====")
    # display(use_df.style.format("{:.4f}"))
    print(use_df.to_string())
else:
    print("Modelo inviável ou sem solução.")

```

Set parameter OutputFlag to value 1

Gurobi Optimizer version 12.0.2 build v12.0.2rc0 (win64 - Windows 11.0  
(26100.2))

CPU model: Intel(R) Core(TM) i9-14900HX, instruction set [SSE2|AVX|AVX2]

Thread count: 24 physical cores, 32 logical processors, using up to 32 threads

Optimize a model with 2 rows, 2 columns and 4 nonzeros

Model fingerprint: 0x5f993c14

Coefficient statistics:

Matrix range [1e-01, 3e-01]

Objective range [7e+00, 8e+00]

Bounds range [0e+00, 0e+00]

RHS range [8e+01, 9e+01]

Presolve time: 0.00s

Presolved: 2 rows, 2 columns, 4 nonzeros

| Iteration | Objective     | Primal Inf.  | Dual Inf.    | Time |
|-----------|---------------|--------------|--------------|------|
| 0         | 2.2700000e+31 | 2.833333e+30 | 2.270000e+01 | 0s   |
| 1         | 2.8080000e+03 | 0.000000e+00 | 0.000000e+00 | 0s   |

Solved in 1 iterations and 0.00 seconds (0.00 work units)

Optimal objective 2.808000000e+03

===== LUCRO ÓTIMO =====

z\* = 2,808.00 (unidades monetárias)

===== PLANO DE PRODUÇÃO (x1, x2) =====

Produto 1 (x1)      360.0

Produto 2 (x2)      0.0

===== USO DE RECURSOS =====

|              | Uso (h) | Capacidade (h) | Folga (h) |
|--------------|---------|----------------|-----------|
| Montagem (h) | 90.0    | 90.0           | 0.0       |
| Teste (h)    | 45.0    | 80.0           | 35.0      |